

A Quantum Algorithm to Verify or Disprove the Collatz Conjecture

M. Chowdhury

3rd May 2025 . Draft version.

Abstract

The Collatz Conjecture (CC) is a famous unsolved problem in number theory. The CC has been verified to large integers but there may be an counterexample to it for larger integers outside tested with present technoloy. We give a new way to disprove the Collatz conjecture using our new quantum algorithm (which is more efficeint than all known classical method) this is the best potential way, to date, to disprove CC if a suatably large quantum computer can be built to find the possible counterexample.

Our algorithm is more efficient regarding space -but time complexity is not greatly different to classical-but it is a certainlly a different way to solve the problem using a qumatum algorithm.

So we present the first quantum agorithm to disprove the CC, We present a quatum algorithm that gives a signifciant speed up compared to known classic CC verification algorithms. The CC is a well known unsolved problem in nuumber theory.We describe a first quantum algorithm for the Collatz Conjecture (also known as the $3x+1$ problem) and it Generalised Collatz conjecture. The best classical algorithms to verify CC up to some postive integer a has best possible running time greater than $O(a)$. Our quantum algorithm uses an oracle by Grover = $O(\log(a)) * O(Sqrt(a))$ time complexity and $O(\log(a))$ space complexity to verify the CC for all integers up to a . Our quantum algorithm is significantly faster and uses less memory than all known classical algorithms. The success probability of increases rapidly the larger the domain the CC is verifed over. If a cycle counter example, of a certain form exists, to the CC exists which is a cycles other 1,2,4,1, then our quantum algorithm can find the counterexample with high probability. We think our quantum algorithm is interesting as the Collatz Conjecture has been an open problem for many years.

1 Introduction and a New Reformulation of the Collatz Conjecture bitwise

The famous CC (Collatz Conjecture) first proposed in 1937 [3], is the following problem. Given any $n > 0$ do all values eventually converge to 1 according to the iteration

$$\begin{aligned} f(x) &= x/2 \text{ if } x \text{ is even} \\ &= 3x + 1 \text{ if } x \text{ is odd} \end{aligned} \tag{0}$$

with initial value $x = x_0$

The above as the Collatz iteration. We write f^i for the i th iterate of $f(n)$. Then for a numerical example, with the initial value is $s = 10$, we have $f^0 = s = 10, f^1 = 5, f^2 = 16, f^3 = 8, f^4 = 4, f^5 = 2, \text{ and } f^6 = 1$. The best theoretical results are "near proof" such as the paper in [1] where it is proved the Collatz conjecture converges with a logarithm density measure.

The CC is then prove that for all positive n the iteration eventually gets into the cycle 1, 4, 2, 1- we refer to this cycle as C . In this manuscript we refer to cycles as periods too, so C can be described as a period defined by the integers 1, 2, 4.

The conjecture has been numerically verified to be true up to $19 * 2^{58}$ [3]. Because of such large integers, with the possibility of large cycles, involved in our opinion a quantum computer program is suited to verify the conjecture and generalisations and variants of this conjecture. Thus the purpose of this paper is to describe how to use a quantum computer for giving numerical evidence for the CC and GCC. In general, there is no classical algorithm that will verify for n in polynomial time, that the CC is true for that n . There are of course polynomial time verification for certain types of integers for n -such as when n is a power of 2. Which leads to the obvious algorithm for verifying the up to some integer R , compute $f(x_0)$ for every x_0 not a power of 2 and $1 \leq x_0 \leq R$ until f converges to 1 for some iterate. This obvious algorithm has worser than $O(R \log(R))$ expected time complexity and expected $O(\log(R))$ space complexity. This obvious algorithm is almost a brute force algorithm except for the power of 2 check. Like the brute force algorithm it computes redundant value since it computes the Collatz orbit for even starting values. A more sophisticated algorithm is Collatz sieve methods which are used currently as the basis to make the largest known verifications of the conjecture. These sieve methods, which are a family algorithms, have a space complexity worser than $O(R)$ and time complexity worser than $O(R \log R)$; the time complexity is basically, we sketch the proof here for a barebones Collatz sieve,

because it needs a loop to check if all numbers up to R , which takes time $O(R)$, have converged then it needs about $\lfloor \log_2(R/5) \rfloor - 1$ multiplications to show that all initial values, $x_0 > 5$, which are multiples (where the multiple is a power of 2) of 5, less than $R + 1$ converge to 1. Hence $O(R)O(\lfloor \log_2(R/5) \rfloor - 1) = O(R \log(R))$ which is the time complexity for this family of algorithms. But there are more operations in the sieve method so it is worse than $O(R \log(R))$ time complexity. The above is only a sketch because current verification methods are refinements of the barebones Collatz in the sketched proof above; the refinements do not change the time complexity to be better than $O(R)$.

Our quantum algorithm has space complexity $O(\log(R))$ and time complexity $O(R)$ which is clearly faster than the obvious algorithm above and algorithms based on sieve methods described above. Note that we cannot give a deterministic upper time and space complexity bound for any classical algorithm to verify the CC nor can we give such an upper bound for our quantum algorithm basically because it is equivalent to proving CC. But for verifying the CC is true on some set of integers our algorithm has better time complexity and space complexity compared to any other classical algorithm.

Now define $LSB(x)$ to be the least significant bit of a function [1]. We defined the binary expansion of Z as

$$Z = Z_{L-1}Z_{L-2}...Z_0$$

where L is the binary length of Z and $Z_{L-1}, Z_{L-2}, \dots, Z_0$ are the binary digits of Z . Assume Z is even then $LSB(Z)$ is the smallest value of E such that the E th digit Z_E in the binary expansion of Z such that all digits to the right are all zero. If Z is odd then $LSB(Z) = 0$. We then discuss the advantages of this new form.

For example $LSB(7)=0$, $LSB(6)=1$, $LSB(20)=2$. Then define

$$LSB'(x) = 2^{LSB(x)} \quad (1)$$

in LSB' words this gives the largest power of 2 that divides x . From the integer $LSB(x)$ LSB' can be expressed with bitwise logical operation (as shown in the appendix). Because it is expressed using bitwise logical operations it means it can be translated into a quantum circuit, using the fact that there are quantum analogues of classical logic circuits, this shows that the reformulated collatz conjecture given by 2 below can be computed in quantum polynomial time and space complexity (we sketch proof in the appendix).

Then we have the following new reformulation of the collatz conjecture and this new form which is required for our quantum algorithm. Given and x_0

$$x_{n+1} = 3x_n + LSB(x_n) \quad (2)$$

converges to a power of 2. $LSB'(x)$ be expressed analytically . The intuition for this since it is a periodic function with different periods for different congruence classes on the integers it, therefore it can be expressed as a sum involving roots of unity, and so it can also be expressed as a function of a discrete Fourier transform equation 3, and thus analytically continue our new reformulation of CC to the complex number. Because we can express $LSB(x_n)$ as

$$LSB'(x_n) = \lim_{p \rightarrow \infty} \sum_{K=0}^{K=U} \left(-\frac{1}{2}\right)^K \left(\frac{1}{4}\right)^p \left(e^{\frac{i}{2^K} \pi x_n} - 1\right) \left(e^{-\frac{i}{2^K} \pi x_n} + e^{\frac{i}{2^K} \pi x_n} - 2\right)^p \quad (3)$$

where U is any constant that is greater than the binary length of x_n . In other words for suitable large p in a concrete implementation we can use the following where $LSB''(x_n, p)$ is an approximation of $LSB'(x_n)$

$$LSB''(x_n, p) = \sum_{K=0}^{K=c} \left(-\frac{1}{2}\right)^K \left(\frac{1}{4}\right)^p \left(e^{\frac{i}{2^K} \pi x_n} - 1\right) \left(e^{-\frac{i}{2^K} \pi x_n} + e^{\frac{i}{2^K} \pi x_n} - 2\right)^p \quad (4)$$

Note that taking p to be a power of 2 means we may use the fast binary exponentiation algorithm to compute it.

since $x_n \geq \lfloor \log_2(x_n) \rfloor$ we can take $U = x_n$. Or a formula with less terms we can take $U = \lfloor \log_2(x_n) \rfloor$. Or we can take a large enough $U = c$ where $c > \lfloor \log_2(x_n) \rfloor$ for all n ; c would have to be larger than the binary length of iterations given by 3. Finding a large enough c is equivalent to solving the CC. We now have the following reformulation of the CC (from substituting 4 into 2) which is suitable form of the conjecture that we can use in our quantum algorithm.

The CC is equivalent to the following new reformulation. For suitably large p and , for any integer x_n

$$x_{n+1} = 3x_n + LSB''(x_n, p) \text{ is expressed as 5.1 below} \quad (5)$$

$$\text{defined as } z(x, p) \quad (5.1)$$

$$x_{n+1} = 3x_n + \sum_{K=0}^{K=c} \left(-\frac{1}{2}\right)^K \left(\frac{1}{4}\right)^p \left(e^{\frac{i}{2^K} \pi x_n} - 1\right) \left(e^{-\frac{i}{2^K} \pi x_n} + e^{\frac{i}{2^K} \pi x_n} - 2\right)^p$$

the iteration 5.1 converges to near a power of 2; that is for any choice of x_n , and arbitrary small positive ϵ there exists n, m, p such that

$$|x_n - 2^m| < \epsilon$$

for some n, m ; where $||$ means the magnitude of the complex number as usual. So we have expressed equation 5 as equations 5.1. Because, our equation above, analytically continues, with C , it is defined for an arbitrary complex numbers not just restricted to the set of natural number as in the original conjecture. The idea of extending the original Collatz iteration was given in [1] as

$$f(z) = \frac{1}{4}(2 + 7z - (2 + 5z) \cos(\pi z))$$

where z is restricted to real numbers for each iteration; the equation above occurs in [1] when the CC iterates are continued to the complex plane.

For example if $x_0 = 10$ and $p = 30$, $m = 5$, $c = 30$, $\epsilon = \frac{1}{2}$ then $n = 1$ since

$$\begin{aligned} |x_1 - 2^5| &= |32.059 + 2.4346 \times 10^{-2}i - 32| \\ &= (0.059)^2 + (2.4346 \times 10^{-2})^2 \\ &= \sqrt{4.0737 \times 10^{-3}} \\ &= 6.3826 \times 10^{-2} \text{ hence} \\ 6.3826 \times 10^{-2} &< \epsilon \end{aligned}$$

Our equation 5.1 gives an insight to why the behaviour of the Collatz function is not easy to analyse, it is a analytical complex iteration on the complex plane that is a "fractal type iteration"; the link between fractals and CC was noted before in [1]. Taking an idea from [1] it would be interesting to investigate the tools of complex analysis and fractal dynamical systems applied to [1] to analyse it. Another advantage of our reformulation, apart from the new reformulation given by 5,5.1 is that it transforms the CC sequences into monotonic increasing sequences i.e. we always have

$$x_{n+1} > x_n$$

in equation 2 and 5.1, this seems to be simpler to analyse, as it looks less random, than the standard collatz sequences given by equation 0-which have "up and downs" that behave in a "pseudo random" way. Another possible advantage is that it shows that if we can get some sort of representation $LSB'(3x_n + LSB'(x_n))$ as a non-nested expression in terms of $LSB'(x_n)$ then immediately this expression for the n th term of as a non-nested expression in terms of $LSB'(x_n)$ thus this may give a way to prove the collatz conjecture. Abbreviating $LSB'(x)$ to $L(x)$ we have and x_0 to x below we have

$$\begin{aligned}
f^0(x) &= x = x_0 \\
f^1(x) &= 3x + L(x) \\
f^2(x) &= f(f(x)) = 9x + 3L(x) + L(3x + L(x)) = 9x + L(f^1(x)) + 3L(x) = 3(f^1(x)) + L(f^1(x)) \\
f^3(x) &= f(f(f(x))) = 27x + 9L(x) + 3L(3x + L(x)) + L(9x + L(3x + L(x)) + 3L(x)) \\
&= 27x + 3L(f^1(x)) + 9L(x) + L(f^2(x)) \\
&= 9f^1 + 3L(f^1(x)) + L(f^2(x)) = 3f^2(x) + L(f^2(x))
\end{aligned} \tag{12}$$

An observation that we will use in the next section on the quantum algorithm is that the classical function f^n takes at most $O(2^n)$ terms and binary operations, as the above examples demonstrates, hence if n grows slowly such as $n = O(\log_2(x_0))$ then we need at most $O(n)$ terms to evaluate f^n .

Similary to equation 5.5.1 we also have

$$x_{n+1} = 3x_n + LSB(x_n) \text{ is expressed as 6.1 below} \tag{6}$$

$$x_{n+1} = 3x_n + \lim_{p \rightarrow \infty} \sum_{K=0}^{K=U} \left(-\frac{1}{2}\right)^K \left(\frac{1}{4}\right)^p \left(e^{\frac{i}{2^K} \pi x_n} - 1\right) \left(e^{-\frac{i}{2^K} \pi x_n} + e^{\frac{i}{2^K} \pi x_n} - 2\right)^p \tag{6.1}$$

Equation 6.1 shows a new connection iterates of the Collatz iteration and dynamical systems, roots of unity (better understood area of mathematics); as mention above we can take $U = x_n$ in above. Roots of unity, of course, appear in Galois theory. So it maybe useful to study some symmetry group assocted with the above equations. Consider the stopping time formula given in [2]. We sketch the proof that equation 5.1 can be implemented efficiently on a quantum circuit. The limit of equation 5.1 is equation 6.1, and the limit of equation of 6.1 i equation A2, which we prove can be implemented efficiently on a quantum circuit

$$\frac{\sigma_\infty(x_n)}{\ln n} = \gamma_\infty(x_n) \tag{7}$$

is the stopping time ratio given [locate ref] conjeured that $\gamma_\infty(x_n)$ converges to some positive constant $n \rightarrow \infty$; proving this is equivalent to proving the CC. In other words the number of iterations, reffered to as t , required to converge for any starting value x_0 is

$$t = \lfloor \eta \ln(x_0) \rfloor \tag{8}$$

for some positive constant η . We referred to the conjecture above involving $\gamma_\infty(x_n)$ as CC'.

To summarise the advantages of our new reformulation of the CC in 2,5.1,6.1,A2

- Equation A2 leads to a provably polynomial time and polynomial space quantum circuit
- Equation 5.1,6.1 show a new connection with a fractal iteration type equation (see [1] for related work)
- If we can express $L(3x + L(x))$ in another form e.g.

$$L(3x + L(x)) = \sum_{\forall i} a_i L(x), a_i \text{ are constants}$$

and using above expressions, such as given by 12, then it may lead to a new insight of why some starting values eventually fall into the cycle with the integers 1,4,2. The behaviour of $L(3x + L(x))$ is related to fractal iterations.

- Equations 5.1,A2 are an essential part of our quantum algorithm given in the next section
- Equation 5.1 above can be written with bitwise operation, as shown in appendix, and thus classical logic gates. This has a natural translation into a quantum circuit. The well known formulation in the iteration 0 above, cannot be directly translated into a quantum circuit in this way and so this is an advantage of our new reformulation above.

2 On a Quantum Algorithm For Verify the Collatz Conjecture

We will modify GA (Grover's algorithm) and use it to verify the CC in our quantum algorithm. The new contribution in this paper is to apply GA to CC and derive a new quantum algorithm. For clarity, firstly we give an exact version of this algorithm then an near as desired approximate version. We can think of this algorithm as a barebones quantum algorithm which is the basis more complicated algorithms; our barebones algorithm is not a brute force algorithm-one reason is it tests odd integers (and no even integers) for the starting values in CC. We give a barebones version for clarity. To be more precise our quantum algorithm

simultaneously checks two conjectures CC' and CC. It assumes CC' is true on the domain of odd integers tested if it turns out to be CC is false for some integer found by our quantum algorithm then this may mean CC' may be false in the sense that η is larger than expected. Because it only tests the conjecture on odd integers it is not a quantum version of the classical brute force algorithm (which checks the redundant case of the even starting values). So this means we modify values of η and verify the CC again on the same domain in this case. Our quantum algorithm checks for convergence applied to CC. The GA type algorithm below is repeated a constant number of times T this is because Grover's algorithm is not an exact algorithm; however for the larger values we are verifying the CC or CC' conjectures for it increases the probability to near 1 that GA gives the correct answer. For each T we record the result of the algorithm -by whatever is the most frequently occurring measured answer is considered as evidence of the correct answer.

2.1 A Quantum Algorithm to Disprove the Collatz Conjecture

Repeat the following algorithm uses a modified Grover algorithm which is repeated a T number of times for some suitably large constant T the "average" answer over T iterations is considered to be correct answer and the output of the algorithm. For each run the measured value is r_i . From the central limit theorem a good value of T is greater than 30 as this gives a good estimate for the arithmetic mean. GA is a probabilistic algorithm regarding its output and it finds the correct answer with a failure rate that exponentially decreases the larger the domain to GA is.

Input: The value α to define the number set of odd consecutive integers on the domain $d = \{1, 3, 5, \dots, 2^{\alpha+1} + 1\}$ for $\alpha > 2$ to verify the CC or CC'. Value of η' which is the maximal feasibly computable classical value. Decide on a value of T

Output: There are 4 possible outputs,

Output i) CC is true on $[1, 2^\alpha]$ with high probability.

Output ii) CC is false on $[1, 2^\alpha]$ because a new cycle is found and hence the CC is false.

Output iii) CC' is false on $[1, 2^\alpha]$

Output iv) Unclassified behaviour output. **Assume** for now, for clarity, that there is at most one counter example exists in d . CC may be false with an initial value x_0 with the possible reason that the trajectory of x_0 is divergent or converges to a new cycle or it converges to the known cycle (though it will need classical analysis to determine this)-this is still an interesting output.

The failure probability, p_{fail} , is the probability of finding a counter example, if it exists which is $p_{fail} = (\frac{1}{2^\alpha})^T$. Thus the algorithm gives the correct answer with probability $p_{success}$ is

$$p_{success} \geq 1 - (\frac{1}{2^\alpha})^T$$

this is because Grover Algorithm would need to fail for all T runs and a possible counter example can be deterministically verified. Note that when there is no counter example to be found in d , even though Grover's algorithm will "fail", this "fail" successfully verifies that no solution in d is found.

Define the oracle as

$$O(t, w) = \begin{cases} 0 & \text{if } f^t(2w + 1) \in \{Q\} \\ 1 & \text{if } f^t(2w + 1) \notin \{Q\} \end{cases} \quad (9)$$

where Q is a set of integers, of a cycle, the function f eventually converges to so for the Collatz problem $Q = \{1, 2, 4, 8, 16, 32, \dots\}$. A key observation is the Collatz function f is expected to take $t = O(\log(x))$ iterations by equation 8 for some arbitrary positive odd integer w ; obviously we need only to verify CC for odd integers. For choice of f we can use equation A given in the appendix. So the maximal running time occurs when x is so it linear time. In words this means if there is a counter example to CC then the oracle is 1 otherwise it is 0. **Assume** for now, for clarity, that there is at most only one value such that $O(t, w) = 1$. Decide on a constant η' , where $\eta' > \eta$, and an integer D which are fixed at all times in all steps in the algorithm.

Step 0. If all T runs have been computed then compute the estimates for the average value $\check{r}$ of the results as

$$\sum_{i=1}^T \frac{r_i}{T} = \check{r}$$

if $\check{r}$ is close to $2^{\alpha-1}$ (the arithmetic average value of the domain or $\frac{2^\alpha}{2}$) then the output is that no counter example to the CC or CC' was found. If $\check{r}$ is not close to $2^{\alpha-1}$ then $\check{r}$ is an estimate is an unclassified for the behaviour of the trajectory generated by $\check{r}$. The output is $\check{r}$ which is an unclassified output.

Step 1. Set the boolean function $m = O(t, w)$ as the oracle in Grover's algorithm.

Step 2.

GA Step i. initlist the register $|a\rangle$ to $|0^n\rangle$ and the work qubit w to $|1\rangle$

GA Step ii. For testing CC and CC' for d we apply the $n+1$ Hadamard gates to get the state initial input state to GA

$$I = \sum_{c=0}^{c=-1+2^\alpha} \alpha_c |c\rangle$$

not that this algorithm tests for x_0 for odd or even;we test it on even integers for expressing the clarity of this algorithm but obviously we only need to test on odd integers.

GA Step iii) Apply the Grover iteration G then increment the loop counter up to H .

$$G|a\rangle \rightarrow |a\rangle$$

Do the Grover iteration a constant number of time H . Where

$$H = \text{Round}\left(\frac{\pi\sqrt{N}}{4} - \frac{1}{2}\right)$$

GA Step iv) At this point,just before measurement, if counter example to the CC exists then it will be mesured with high probability. Now measure register $|a\rangle$ for the answer x .

Step v). This is a classical post processing step that uses the answer x just measured in ster iv above. If a counter example to the CC is not found we have $x_0 = x$ such that has converged i.e. it means $f(x_0) \in Q$. Note that $f(x_0) \in Q \iff \log_2(f(x_0)) \in N^+$ which can be decided in polynomial complexity. We can classically compute $O(t, x_0)$ to verify that $f(x_0) \in Q$; which is equivalent the standard obvious algorithm to do this in polynomial space and polynomial time complexity. This means no counter example to the CC is found and we do not need to consider the point below. For this run of the algorithm (out of T runs) it is noted that no counter example was found. So we can say from [3] that up to 2^{57} that the algorithm will find that $f(x_0) \in Q$ since we know, under current knlowge, no such example to the CC exists. Since $f(x_0) \in Q$ then stop the **output of the algorithm is a counterexample to CC was not found, the output is $r_i = x$,do the next run to compute r_{i+1} , of the T runs,the algorithm.** If $f(x_0) \in Q$ go to step Step iv-a). that is no counter example is found the oracle does not mark any states and consequently the Grover amplitude amplification process does not work.

- Step iv-a) If a counter example to the CC is found or counter example to the CC' , then it will be observed with $(1 - \frac{1}{2^\alpha})$ chance

or greater, we have x_0 such that it not converged i.e. it means $f(x_0) \notin Q$. Since the failure probability of GA is $\frac{1}{2^\alpha}$. In other words if it is measured that $x_0 = x$ and we can classically compute $O(t, x_0)$ in to verify that $f(x_0) \notin Q$; which is equivalent the standard obvious algorithm to do this in polynomial space and polynomial time complexities. GA is not an exact algorithm but the failure chance of it decreases rapidly the large α is. There are still possibilities to what the measured value means-to decide firstly go to Step iv-b)

- Step iv-b) *Counter example to the CC' is found.* This means that η is larger than previously conjectured, so increase η by some fixed constant to get η' , and if we now have

$$f^{\lfloor \eta' \ln(x_0) \rfloor}(x_0) \in Q \quad (10)$$

then we modify the CC' conjecture so it conjectures a larger value for the constant see [2]. In other words it still takes a polynomial number of iterations to converge to 1. If 10 turn out to be true then stop the **output of the algorithm is that a counter example to CC'is found, do the next run, stop algorithm and do a new set of runs (it means start computing from r_1 again to recompute the average) with the updated value η' .** If 10 is not true at this point then we have may found a new cycle to test this possibility go to Step iv-c) .

- Step iv-c) counter example to the CC is found which is a new cycle different from the cycle containing 1,2,4. It is known that cycles length δ are of the form [3]

$$\delta = 301994a_1 + 17087915a_2 + 85137581a_3 \text{ for } a_2 > 0, a_1a_3 = 0 \\ \text{and } \delta \geq 114208327604 = (17087915 * 361) + (85137581 * 1269)$$

So we check for a cycle of length δ by selecting a random $a_1 = 0, a_2 \in [361, 1280], a_3 \in [1269, 1280]$; the upper bounds of these intervals are arbitrary but they still show the algorithm runs in linear complexity . If a new cycle is found stop. So we are testing for cycles of a specific length by evaluating

$$f^\delta(x_0) \stackrel{?}{=} f^{2\delta}(x_0) \quad (11)$$

Which is classically ncomputable in polynomial time and polynomial space since the bounds on a_1, a_2, a_3 are fixed by 1280. **If 11 is true then the output of the algorithm is th Collatz cojecture is disproved , output this as the counte example counter example, stop and**

do not do the remaining runs . Another way to find the cycle is by using [0] and computing until a repeat up to the chosen bound **using this to disprove**. If it fails assume at this point we have possible evidence that it divergent trajectory; though this measured value an still be a convergent orbit (if we were able to use a greater η') , a new cycle (if it lies outside the types of cycles tested) or a divergent orbit. Hence the measured answer x is an unclassified output. **So the output of the algorithm, is $r_i = x$, do the remaining T runs the algorithm which are expected to give the same possible counter example as output**. We see that the time complexity is dominated by the computation of the oracle (which is of linear time complexity); but because the oracle is computed in a quantum parallel superposition for all input values (i.e. elements of d) to test- this is a fact that makes it faster than all known classical algorithms for verifying the CC.

References

- [1] Marc Chamberland, A Continuous extension of the $3x+1$ problem to the real line, *Dynam. Contin. Discrete Impuls. Systems* 2: 4 (1996), 495-50
- [2] A. V. Kontorovich, J. C. Lagarias, Stochastic Models for the $3x+1$ and the $5x+1$ problems, in *The Ultimate Challenge: The $3x$ problem.*, Amer. math. Soc. Providence RI 2-0 pp. 3-88
- [3] Collatz Conjecture, Wikipedia, www.wikipedia.org/Collatz_conjecture
- [4]

3 Appendix

$LSB'(x)$ is expressed in the language of C++ notation , so as usual $|, \&, ^$ are logic operations on binary digits in C++, and $<<$ is the left shift operation in C++. Define $K(x, L)$ (for clarity) follows

$$K(x, L) = x \mid ((2^L - 1) \& (x < 1)) \mid ((2^L - 1) \& (x << 2)) \mid ((2^L - 1) \& (x << 3)) \dots \mid (((2^L - 1) \& (x << L)))$$

$K(x, L)$ contains L terms of the form $((2^L - 1) \& (x << s))$ for some natural number $s \in [1, L]$. Then

$$LSB'(x) = ((2^L - 1) \& (K(x, L) < 1)) ^ K(x, L) \quad (A)$$

which can of course be expressed without $K(x, L)$ -but we use $K(x, L)$ for clarity. L is a suitably large integer and it to be greater than the

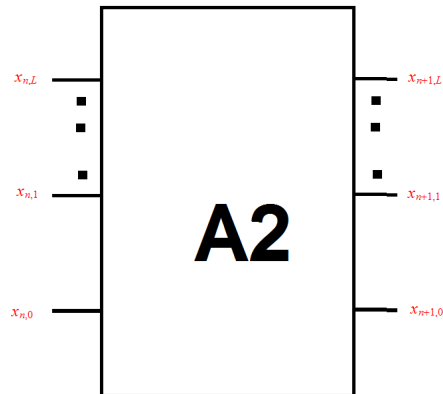
binary length of the largest integer in the collatz trajectory of x_0 . Thus the Collatz iteration can be expressed in terms of logical connectives. Which is

$$\begin{aligned} x_{n+1} &= 3x_n + ((2^L - 1) \& (K(x_n, L) << 1)) \wedge K(x_n, L) \\ &= (x_n << 1) + x_n + ((2^L - 1) \& (K(x_n, L) << 1)) \wedge K(x_n, L) \end{aligned} \quad (\text{A2})$$

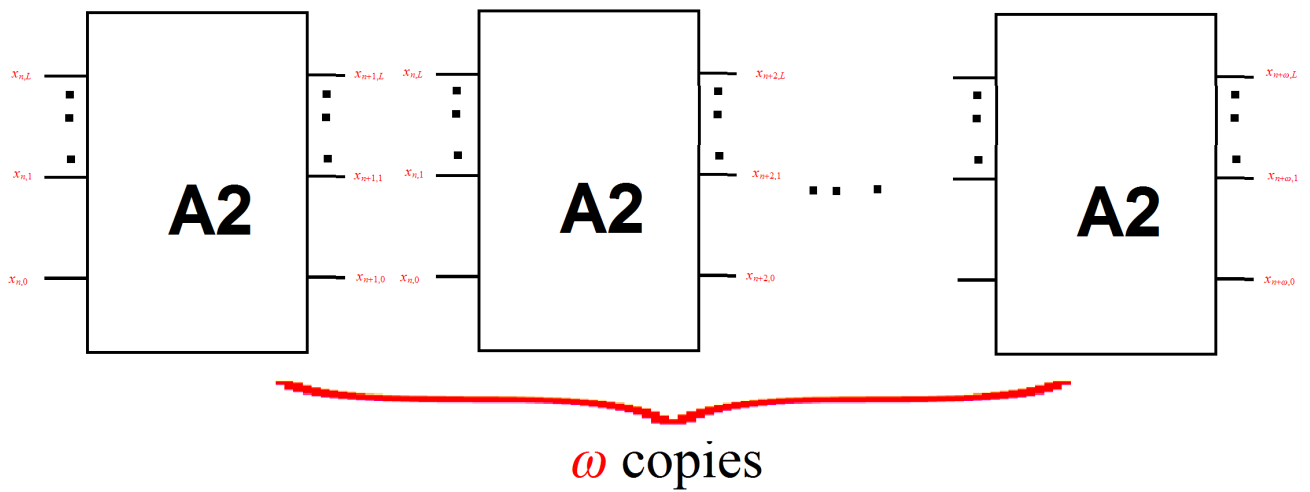
In this form of it can be directly translated into assembly language, because all classical CPUs support and, or, xor on integers as assembly language instructions, with the direct implementation an N -bit CPU can test the collatz conjecture for the trajectory values starting with x_0 bounded above by the maximum word size of the CPU which is $2^N - 1$. Assembly language is faster than a higher level language with a suitable implementation of the same code. For testing if the trajectory of x_0 converges to the cycle or a power of 2 a CPU can simply do L comparisons for each integer $1, 2, 3, \dots, 2^L$. It can also be implemented with arbitrary large integer C++ code regarding the binary digit size of integers.

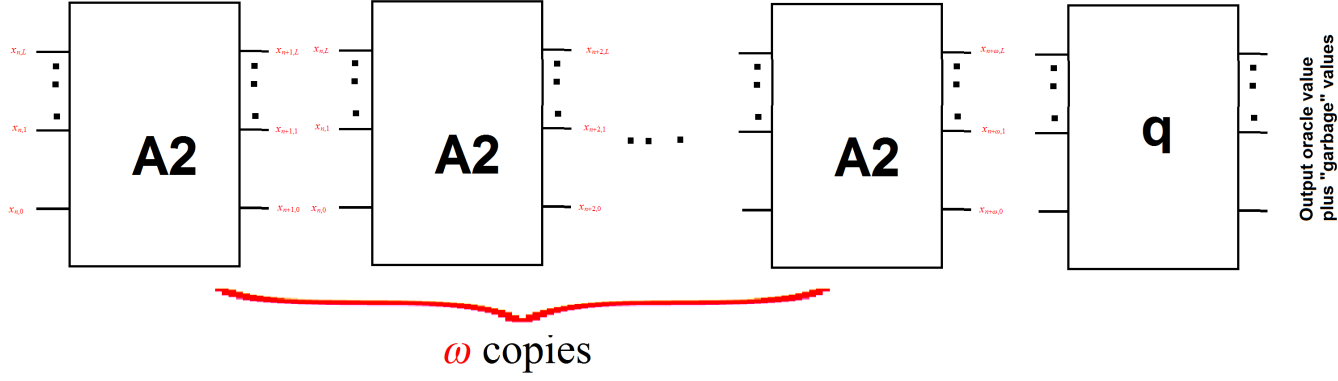
We sketch a proof of why the Collatz iterate can be computed in quantum polynomial complexity using above. As mentioned above $K(x, L)$ contains L terms of the form $((2^L - 1) \& (x << s))$ for some natural number $s \in [1, L]$ $L = O(\log(x))$. Because it is expressed using bitwise logical operations it means it can be directly translated into a quantum circuit, using the fact that there are quantum analogues of classical logic circuits, therefore the quantum version of equation A can be computed in quantum polynomial time and space complexity. By substituting equation A into the system of equation 12 we see that the Collatz iterates takes at most linear $O(x)$ time and space classically in the worst case, hence it shows our quantum algorithm (the oracle) runs in linear time in worst case.

The oracle is constructed as follows (a sketched out description—with rough diagrams below—but this has enough detail it can be filled in by an expert reader). The well known Collatz iteration, zero above, is rewritten as equation 5, which is rewritten as equation A or A2 above. The diagram below shows one iteration of A2 as a quantum circuit which is equivalent to one iteration of the collatz conjecture where $x_{n,L} \dots x_{n,1} x_{n,0}$ are the qubits that make up the integer value of the iteration x_n and where $x_{n,L}$ is the most significant qubit of x_n and x_0 is the least significant qubit of x_n and where L is the bitlength of the number x_n here.



Thus ω iterations will be (as the red bracket in the diagram below shows)





and it is expected ω is polynomial from above. Then the above implements part of equation 9 i.e. it computes $f^t(2w + 1)$ with $t = \omega$. Therefore to fully implement equation 9 the circuit q which tests if $f^t(2w + 1) \in \{Q\}$ the circuit for the oracle is thus

Of course there may be other ways to achieve the same result using the ideas in this paper to give alternative circuits for the oracle above.